백엔드 개발자를 위한 최소 AI 지식

기준일 2026년 10월 14일. **블록당 3~5분**. 가상 예시이며 기출·예상문항이나 확정 출제 범위가 아니다. 읽기와 독립 풀이는 다르며 합격을 보장하지 않는다.

이 책의 범위와 읽는 순서

범위	이 책에서 다루는 수준	다음 학습
모델과 LLM	개념·입력·출력의 한계	블록 1~4
데이터와 평가	분할·누수·작은 손계산	블록 5~6
수학과 딥러닝 내부	미분·역전파·attention 수식 제외	기초 커리큘럼에서 확장
GPU와 효율화	HBM·KV cache·양자화 심화 제외	이후 개발에 맞춰 확장

1→6 순서로 읽고 연습한다. 제외한 범위도 출제 여부를 단정하지 않는다. 기초 커리큘럼에서 확장한다.

1 모델은 데이터에서 배운 예측 규칙이다

머신러닝 machine learning, ML은 데이터에서 패턴을 배우는 방법, **모델 model**은 예측에 사용하는 구조다. **학습 training**은 데이터로 내부 값인 **파라미터 parameter**를 조정하는 과정, **추론 inference**은 준비된 모델로 새 입력을 처리하는 과정이다. API에 질문했다고 모델을 학습시킨 것은 아니다.

요청 직전 CPU 사용률처럼 입력에 쓰는 값은 **특징 feature**, 과거 요청의 실제 지연처럼 학습할 정답은 **라벨 label**이다. 입력과 정답으로 배우는 방법은 **지도학습 supervised learning**이다.

정답 없이 비슷한 로그를 묶는 **군집 clustering**은 **비지도학습 unsupervised learning**의 예다.

회귀 regression은 240ms 같은 수치, **분류 classification**은 정상·장애 같은 범주를 예측한다. 분류 점수에 기준값인 임계값을 적용해 범주로 바꾸기도 한다. **생성 AI generative AI**는 보고서 같은 내용을 만든다. 회귀는 오차, 분류는 오탐·누락, 보고서는 내용과 근거를 검증한다. Google ML.

흔한 오해: 실행 성공이면 좋은 답이다. 서버 응답과 결과의 타당성은 따로 평가한다.

확인 질문: 요청의 실패 여부와 지연 ms 예측은 무엇이 다른가? **답:** 앞은 분류, 뒤는 회귀이며 평가도 다르다.

2 LLM은 문맥을 보고 내용을 생성한다

대규모 언어 모델 large language model, LLM은 많은 데이터로 학습해 언어를 처리·생성한다. **토큰 token**은 입력을 처리하는 단위이며 단어·조각·문자 등에 대응한다. 한국어 한 글자나 한 단어와 항상 같지는 않다.

문맥 context은 이번 생성에 전달한 정보, **문맥 창 context window**은 한 번에 다루는 정보의 한도다. 요청·대화·문서·도구 결과 등이 공간을 쓴다. 파일이 PC에 있어도 모델에 전달되지 않았다면 읽었다고 볼 수 없다. DB에 행이 있는 것과 현재 요청이 조회한 것이 다른 것과 같다.

환각 hallucination은 근거 없는 내용을 그럴듯하게 만드는 오류다. 존재하지 않는 로그를 인용하거나 "DB 커넥션 부족이 원인"이라고 단정할 수 있다. 말투는 증거가 아니다.

LLM이 문맥 다음의 토큰을 선택하는 확률과 그 문장이 사실일 확률은 다르다.

온도 temperature는 지원하는 모델에서 생성의 무작위성을 조절한다. 낮은 값이 정확성을 보장하지 않고 0에서도 API 결과가 다를 수 있다. 지원 여부도 확인한다. Claude 용어집.

흔한 오해: 긴 문맥과 낮은 온도면 해결된다. 답을 원문에 대조한다.

확인 질문: 같은 원인을 세 번 말하면 믿는가? **답:** 반복은 입증이 아니다. 실제 로그·확인 결과가 필요하다.

3 지시 검색과 추가 학습은 다르다

프롬프트 prompt는 모델에 전달하는 요청과 지시다. "오류율 계산"보다 "이 API·시간 구간의 500 건수를 전체 요청 수로 나누고 분모를 표시"가 분명하다. 목표·자료·제약·출력·모르는 경우의 처리를 적는다. 소수 예시로 원하는 패턴을 보여주는 **few-shot prompting**도 있다. 프롬프트.

검색 증강 생성 retrieval augmented generation, RAG는 관련 자료를 찾아 문맥에 넣고 생성하는 방식이다. 현행 장애 문서를 로그와 함께 주는 예다. **임베딩 embedding**은 텍스트 등을 숫자 목록인 벡터로 표현해 관련성을 비교한다. 유사도는 사실성·현행성 판단과 다르며 0.95를 정답 확률 95%로 읽지 않는다. 임베딩, 검색.

미세조정 fine-tuning은 추가 데이터로 모델을 학습해 내부 값인 파라미터를 조정한다. 프롬프트는 지시, RAG는 자료 공급, 미세조정은 추가 학습이다. 미세조정도 최신 사실을 보장하지 않는다. Hugging Face.

흔한 오해: RAG면 환각이 없다. 검색 누락·폐기 문서 선택·생성 오류를 나눠 본다.

확인 질문: 폐기 문서가 검색 1위면 적용하는가? **답:** 아니다. 대상·효력·버전부터 확인한다.

4 도구를 쓰는 모델과 실행 계층을 구분한다

구조화 출력 structured output은 필드·타입·허용값 같은 스키마에 맞춰 결과를 받는 방식이다. 유효한 JSON과 스키마 준수도 다르다. `{"error_rate": 0.05}`가 구조상 맞아도 실제 비율이 0.20이면 틀렸다. 구조화 출력.

도구 호출 tool calling은 모델이 함수와 인수를 요청하는 방식이다. 앱이 권한·인수를 검사해 실행하고 결과를 돌려준다. `query_logs` 요청과 DB 조회 성공은 다르다. 타임아웃은 오류 0건이 아니라 실패로 반환한다. 함수 호출.

하네스 harness는 모델의 문맥·도구·상태·권한·반복을 관리하는 실행 계층이다. **워크플로 workflow**는 코드가 정한 경로로 진행한다. **에이전트 agent**는 모델이 목표와 관찰에 따라 다음 행동·도구를 선택하며 반복한다. 고정 경로에도 LLM을 쓰므로 출력까지 항상 같지는 않다. Claude Code, 두 방식의 구분.

SQL 집계→검색→보고서가 고정돼 있으면 워크플로다. 모델이 DB 오류를 보고 추가 조회를 선택하며 가설을 바꾸면 에이전트 구조에 가깝다. 코딩 에이전트로 만든 앱의 흐름도 따로 본다.

흔한 오해: 이름이 Agent면 에이전트다. 행동 선택과 실제 결과의 반영을 본다.

확인 질문: JSON이 맞고 "조회했다"고 말하면 충분한가? **답:** 내용과 실제 호출 결과를 확인한다.

5 좋은 점수인지 알려면 평가 구조를 본다

학습 세트 training set는 학습에, **검증 세트 validation set**는 모델·설정 선택에 쓴다. **테스트 세트 test set**는 선택 후 평가한다. 테스트 점수를 보며 수정하면 그 데이터에 맞춰 평가의 독립성이 약해진다. Google 데이터 분할.

데이터 누수 data leakage는 예측 시 알 수 없는 정보나 평가 자료가 학습·선택에 들어오는 문제다. 요청 전 예측에 종료 후 상태 코드를 쓰는 예다. 값의 변환에 쓸 평균도 학습 자료에서 구해 적용한다. scikit-learn 누수.

가상 장애 감지 100건에서 장애를 맞힌 **참양성 true positive, TP**은 8건, 정상을 장애로 오탐한 **거짓양성 false positive, FP**은 2건이다. 장애를 놓친 **거짓음성 false negative, FN**은 2건, 정상을 맞힌 **참음성 true negative, TN**은 88건이다.

지표	묻는 것	이 예제의 계산
정밀도 precision	장애라고 경보한 것 중 진짜는	TP/(TP+FP) = 8/10 = **80%**
재현율 recall	실제 장애 중 찾아낸 것은	TP/(TP+FN) = 8/10 = **80%**
정확도 accuracy	전체 중 맞힌 것은	(TP+TN)/100 = 96/100 = **96%**
F1 점수	정밀도·재현율의 조화평균	2TP/(2TP+FP+FN) = 16/20 = **0.8**

정확도 96%여도 실제 장애 10건 중 2건을 놓쳤다. 누락과 오탐의 비용에 맞는 지표를 골라야 한다. Google 분류 평가 지표.

흔한 오해: 점수가 높으면 모든 상황에 강하다. 자료의 대표성·누수·오류 유형도 본다.

확인 질문: 경보 중 진짜 비율은? **답:** 정밀도다. 실제 장애를 얼마나 찾았는지는 재현율이다.

6 답을 제출하기 전에 분모와 근거를 확인한다

AI 답변도 백엔드 응답처럼 검사하되 계산과 근거까지 확인한다.

1. **분모:** 두 사용자의 요청이 100건·1건, 오류가 1건씩이면 전체 오류율은 2/101이다. 사용자별 비율의 단순 평균 50.5%와 다르다.
2. **단위:** 초·ms, MB·MiB를 구분한다. 1MiB는 1,048,576바이트, 1MB는 1,000,000바이트다. NIST 단위.
3. **변화:** 10%→15%는 5%p 상승, 상대 증가는 50%다.
4. **출처:** 원문·날짜·현행 여부·대상을 확인한다. 검색됐다고 적용하지 않는다.
5. **주장·실행:** 배포 후 증가와 원인 입증을 나눈다. 실행하지 않은 조회는 성공으로 보고하지 않는다.
6. **개인정보 privacy·권한:** 개인 식별 정보·API 키를 불필요하게 외부 모델이나 공개 파일에 보내지 않는다. 읽기와 변경 권한은 다르다.

외부 자료의 명령으로 행동을 바꾸려는 공격은 **프롬프트 주입 prompt injection**이다. "비밀을 전송하라"는 메모는 권한을 주는 지시가 아니다. 도구·데이터 범위와 시간·호출·재시도 한도를 코드로 관리한다. OpenAI 안전.

흔한 오해: "정확하게 답해"면 검증이 끝난다. 수치·원문·실행·권한을 본다.

확인 질문: 오류가 늘면 자동 롤백하는가? **답:** 원인·권한을 단정하지 말고 적용 규정과 근거를 확인한다.

다시 보는 열두 장의 요약 카드

카드	기억할 한 문장
1 학습·추론	배우기와 사용하기를 구분한다.
2 회귀·분류	수치와 범주를 구분한다.
3 토큰·문맥	단어 수와 토큰 수는 다르다.
4 환각·온도	반복·낮은 온도도 사실 보장이 아니다.
5 프롬프트	목표·자료·제약·출력을 적는다.
6 RAG·학습	자료 공급과 파라미터 조정은 다르다.
7 임베딩	유사도는 사실 확률이 아니다.
8 구조·내용	스키마 뒤에도 내용을 검사한다.
9 도구·하네스	요청·실행·관찰을 구분한다.
10 실행 흐름	다음 행동을 누가 선택하는지 본다.
11 평가	분할·누수·오탐·누락을 본다.
12 제출	단위·출처·권한·개인정보를 본다.

사전 연습 세 가지

1. **20문항:** 아래 명령으로 풀고 오답을 다시 설명한다. 기존 창작 문제의 선택 목록이다. 30분은 연습 예산이며 입력을 강제 종료하지 않는다.

```
.\scripts\study.ps1 -Mode timed
```

2. **공식 체험:** 프로그래머스에서 읽기·AI 답 검토·제출을 경험한다. 체험 조건을 실전 규칙으로 옮기지 않는다.

3. **자료 검증:** 준비도 연습 안내의 RC02를 풀어 문서 효력과 단위를 판단한다. 정답 전에 답을 저장한다. `/prompt`는 초안 기록이며 AI 호출이 아니다.

연습 뒤 "고친 오해와 검증 근거"를 적는다.